

TMF630 REST API Design Guidelines Part 6: JSONPath extensions

TMF630 Part 6

Maturity Level: General Availability (GA)	Team Approved Date: 04-Sep-2025
Release Status: Pre-production	Approval status: Team Approved
Version: 5.0.0	IPR Mode: RAND

Notice

Copyright © TM Forum 2025. All Rights Reserved.

This document and translations of it may be copied and furnished to others, and derivative works that comment on or otherwise explain it or assist in its implementation may be prepared, copied, published, and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice and this section are included on all such copies and derivative works. However, this document itself may not be modified in any way, including by removing the copyright notice or references to TM FORUM, except as needed for the purpose of developing any document or deliverable produced by a TM FORUM Collaboration Project Team (in which case the rules applicable to copyrights, as set forth in the [TM FORUM IPR Policy](#), must be followed) or as required to translate it into languages other than English.

The limited permissions granted above are perpetual and will not be revoked by TM FORUM or its successors or assigns.

This document and the information contained herein is provided on an “AS IS” basis and TM FORUM DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

Direct inquiries to the TM Forum office:

100 Enterprise Drive
Suite 301 #1649
Rockaway, NJ 070866, USA
Tel No. +1 862 227 1648

TM Forum Web Page: www.tmforum.org

Table of Contents

Notice	2
Executive Summary	4
Conventions	5
Keywords to Indicate Requirement Levels	5
Pattern Notation	5
Formatting Conventions for Examples	6
1. Introduction	7
1.1. Use Cases	7
1.2. Collection filtering using JSONPath (" <i>filter</i> ")	11
1.3. Resource partial representation using JSONPath (" <i>fields</i> ")	12
2. JSONPath	13
2.1. What is it	13
2.2. Example JSON Document	13
2.3. Syntax	15
2.4. Examples	22
3. URL / URI Encoding of the JSONPath expressions	37
4. Collection filtering using JSONPath	39
5. Partial resource representation using JSONPath	46
6. Sorting selector	50
7. Notification Pattern – Registering Listener	51
8. Error handling	53
9. JSON Patch	54
References	57
Administrative Appendix	59
Document History	59
Version History	59
Acknowledgments	59

Executive Summary

The “REST API Design Guidelines” document provides guidelines and design patterns used in designing TM Forum REST APIs. The document is organized in multiple parts as follows:

- **Part One:** Practical guidelines for RESTful APIs; naming, CRUD, filtering, tasks, history, notifications
- **Part Two:** Advanced guidelines for RESTful API polymorphism, extension patterns, depth and expand directive, entity RefOrValue
- **Part Three:** Guidelines for extending TMF Open APIs with hypermedia support
- **Part Four:** Advanced guidelines for RESTful APIs; lifecycle management, common tasks
- **Part Five:** JSON Patch extension to manage arrays
- **Part Six:** JSONPath extension
- **Part Seven:** JSON Schema patterns
- **Part Eight:** API Specialization

This part (Part 6) provides information for the development of TMForum OpenAPIs using REST.

It provides recommendations and guidelines, to extend the existing design patterns through the usage of JSONPath across the TMForum OpenAPI REST ecosystem.

Conventions

Keywords to Indicate Requirement Levels

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in [\[RFC8174\]](#).

Pattern Notation

To formally describe patterns, the notation defined in [\[RFC5234\]](#) and extended in [\[RFC9110\]](#) (section 5.6.1) is applied:

- `[ ]` Square brackets enclose an optional element.
- `( )` Elements enclosed in parentheses are treated as a single element whose contents are strictly ordered.
- `/` (forward slash) Elements separated by a forward slash are alternatives.
- `*` (star) preceding an element means repetition without separator between different values; a number preceding the star symbol defines the minimal number of values, defaulting to 0, and a number following the star symbol defines the maximal number of values, defaulting to unbounded; thus, `1*` indicates an unbounded repetition having at least one value.
- `#` (hash) preceding an element means repetition with a `,` (comma) as separator between different values; a number preceding the hash symbol defines the minimal number of values, defaulting to 0, and a number following the hash symbol defines the maximal number of values, defaulting to unbounded; thus, `1#` indicates an unbounded comma-separated repetition having at least one value.
- `" "` One or more symbols enclosed in quotation marks represent the enclosed symbols as literals.

Deviating from the RFC5234 notation:

- `{ }` A string enclosed in curly braces represents a value of an element named by the enclosed string.

For better readability, spaces may be inserted between elements, which do not become part of the pattern's production. Only if enclosed in quotation marks spaces are meant to be part of the pattern production.

Formatting Conventions for Examples

In most examples, attributes occur that normally would be long UUID values; as simplification of the layout (avoidance of unnecessary line breaks), these values are shortened to a length of 4 characters only, followed by two dots:

`bd69..` instead of `bd694f23-627f-4553-a1df-7e79d62001ac`

For the same reason, in examples we abbreviate the leading part of URLs by a placeholder:

`{apiRoot}` instead of `https://mycsp.com/tmf-api`

To increase readability, we do not show the encoded form of URLs, but use the unencoded form. Of course, in a real implementation the URLs must be encoded as per [\[RFC3986\]](#).

Chapter 1. Introduction

The fundamental concept in any RESTful API is the *resource*. A resource is an object with a type, associated data, relationships to other resources, and a set of methods that operate on it.

Resources can be grouped into *collections*. Each collection is homogeneous so that it contains only one type of resource, and unordered. Collections are themselves resources as well.

The diagram below illustrates the key concepts in a RESTful API.

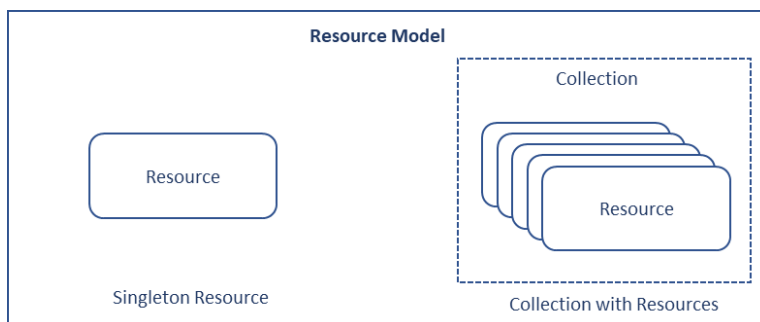


Figure 1: Resource Model

Resources have data associated with them which are represented using JSON.

JavaScript Object Notation (JSON) [\[RFC8259\]](#) is one of the most common REST API specification formats for the exchange and storage of structured data. TMForum OpenAPI Design Guidelines enforce the use and support of the JSON media type across the REST APIs as:

- *The server MUST support "application/json" by default.* – [\[TMF630-5\]](#)
- *REST APIs MUST support the "application/json" media type by default.* – [\[TMF630-4\]](#)

1.1. Use Cases

Considering the resource model, the main use cases are:

- Retrieving a sub-set of resources from a collection based on a set of JSONPath conditions, via the **Filter** directive.
- Retrieving a partial representation from a singleton resource, via the **Fields** directive.

The TMForum OpenAPI Design Guideline provides support for querying resources with (or without) attribute filtering via the design patterns. This document aims to extend by

augmenting the existing design patterns with the JSONPath [\[RFC9535\]](#) query language features.

JSONPath is a standard for querying JSON documents, defined in [\[RFC9535\]](#). Numerous libraries are available for a wide range of programming languages allowing the developers easy access to the features of the query language.

The adoption of JSONPath in the TMForum OpenAPI Design Guidelines is motivated by the desire to fulfill the query gaps in hierarchical multilevel array structures and to provide new query & filtering capabilities, standardized by a common syntax and behavior model.

One scenario, where the collection filtering using JSONPath is required, is in the scenario of the hierarchical query. The hierarchical query is heavily used across different TMForum openAPIs.

In the case of the hierarchical query, each node needs to be evaluated and the conditions to be satisfied.

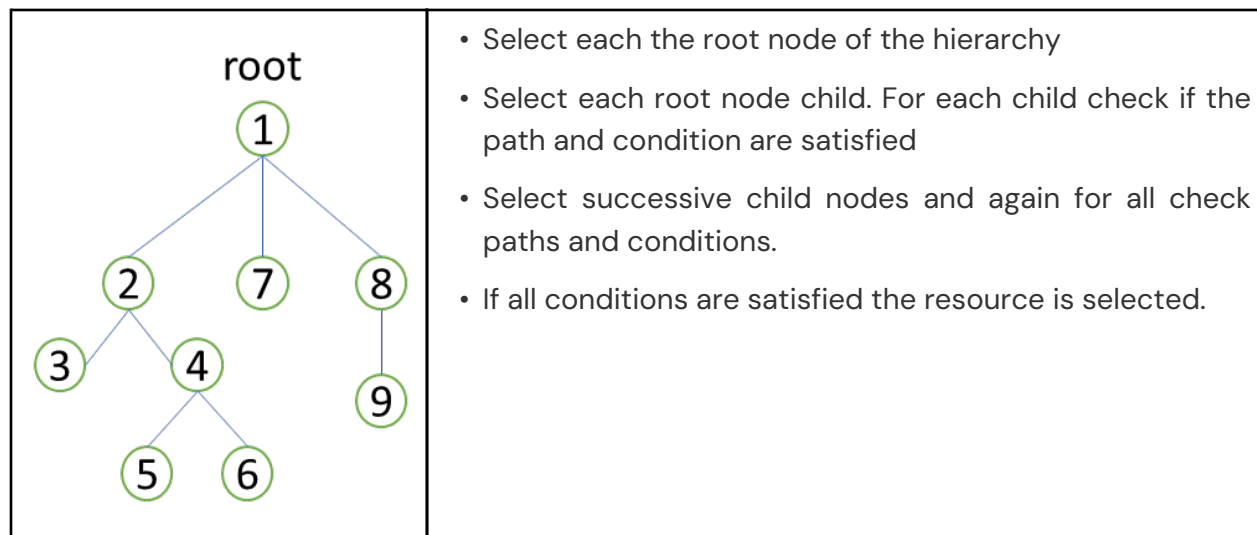


Figure 2: Simple graph

As JSON is a hierarchical structure, let's consider a simple example to demonstrate the limitations of the standard key-value query and the solution proposed through the usage of JSONPath for collection filtering.

Considering the following simple example, for a "Building" collection, where each building has a set of attributes (lift condition) and an array for apartments where the number of rooms are listed.

```
{
  "building": [
```

```
{
  "name": "Babbage",
  "floor": [
    {
      "level": 1,
      "apartment": [ {"rooms": 2}, {"rooms": 3} ],
      "lift": "working"
    },
    {
      "level": 2,
      "apartment": [ {"rooms": 1}, {"rooms": 4} ],
      "lift": "notworking"
    }
  ]
},
{
  "name": "Charles",
  "floor": [
    {
      "level": 1,
      "lift": "notinstalled",
      "apartment": [ {"rooms": 1}, {"rooms": 2} ]
    },
    {
      "level": 2,
      "lift": "working",
      "apartment": [ {"rooms": 1}, {"rooms": 4} ]
    }
  ]
}
]
```

If the request is to select all buildings that have a lift in working condition and there is an apartment with a single room, the query string will look like this:

```
GET /api/building?building.floor.lift=working&building.floor.apartment.rooms=1
```

However, the above query will produce the wrong result, returning both **Babbage** and **Charles** resources, because there is no mechanism in the query string to highlight that there should be an intersection between the two input conditions.

The graphical representation of the above query selection:

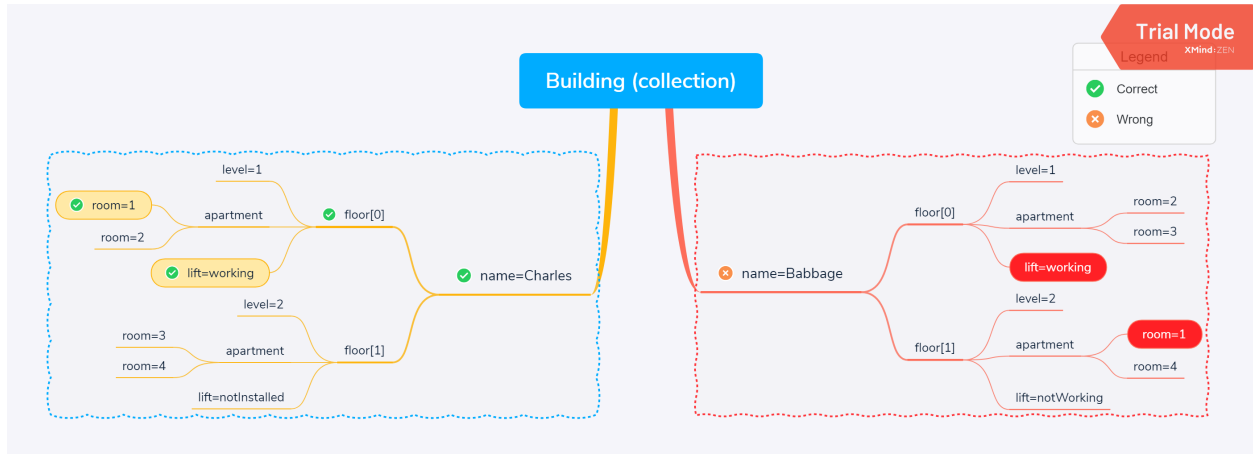


Figure 3: Collection filtering through simple query selection result

As seen above, the resource **Babbage** will be selected when the query should only return the **Charles** resource.

The same JSON resource as above, this time using the collection filtering with JSONPath:

```
GET
/api/building?filter=$.building[*].floor[?(@.lift=="working")].apartment[?(@.rooms==1)]
```

will result only in the "Charles" resource to be selected and returned to the client.

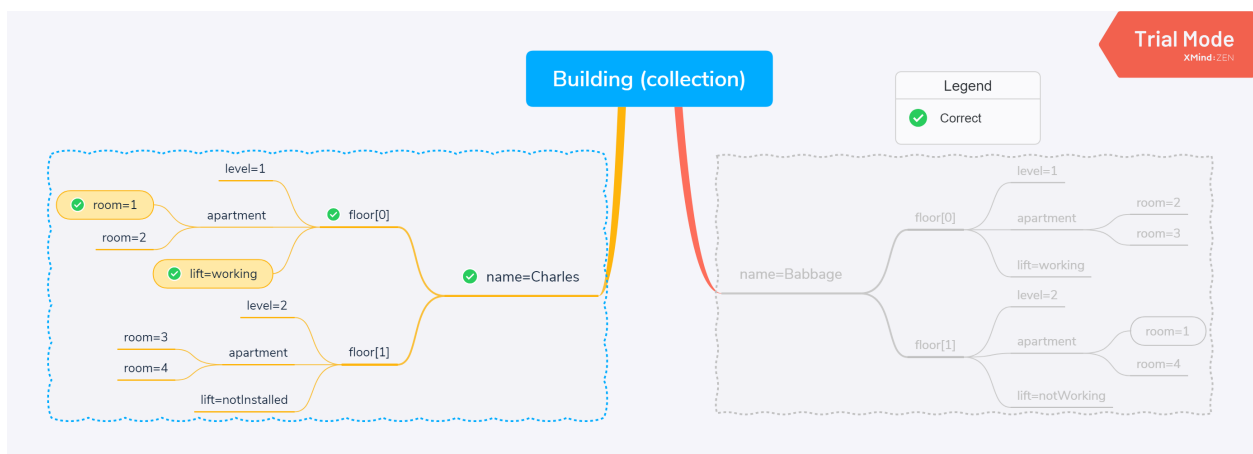


Figure 4: Collection filtering through *filter* selector result



for clarity in the above example the characters [and] are not encoded as they should.

1.2. Collection filtering using JSONPath ("filter")

As per RESTful APIs guideline ([TMF630-1]), **GET** HTTP operation is being used to retrieve either all the resources in a collection (subject to server limitations, pagination, etc.) or to retrieve only a subset of the resources from the collection based on condition(s) filtering.

Collection filtering is done by taking the input condition (Boolean) and applying it against the collection of resources. By doing so, only the resources where the condition is satisfied (true) are selected and then returned to the API client.

Other parts of the TMForum Design Guidelines support only the mechanism of basic filtering which is based on using name-value query parameters on entity attributes (as described in [TMF630-1])

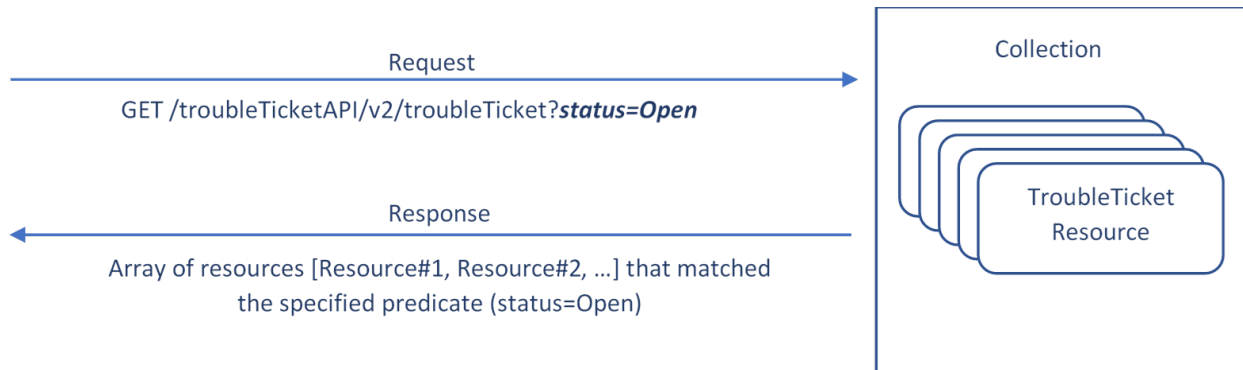


Figure 5: Example of a basic query mechanism

This part of the design guide enhances the collection filtering mechanism through the use of a JSONPath selector called **filter** to provide a flexible way to select a subset of resources from a collection by using the JSONPath predicate selectors and functions. The JSONPath collection filtering provides a standardized way of navigating through complex multilevel resources data models.

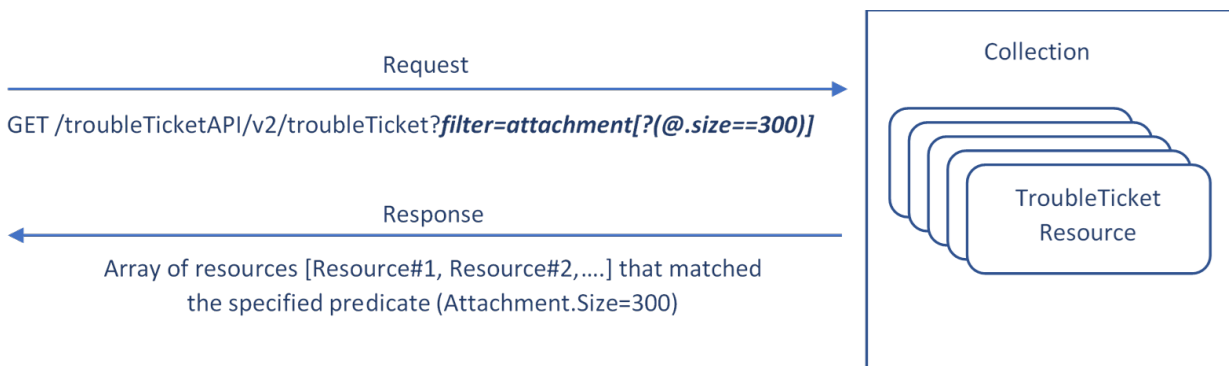


Figure 6: Example of a filter query mechanism

1.3. Resource partial representation using JSONPath ("fields")

This pattern describes how to retrieve a subset of the attributes from an entity in the response using an attribute filtering mechanism. In the current Design Guideline the filtering is done using the selector directive called **fields** – [TMF630-1].

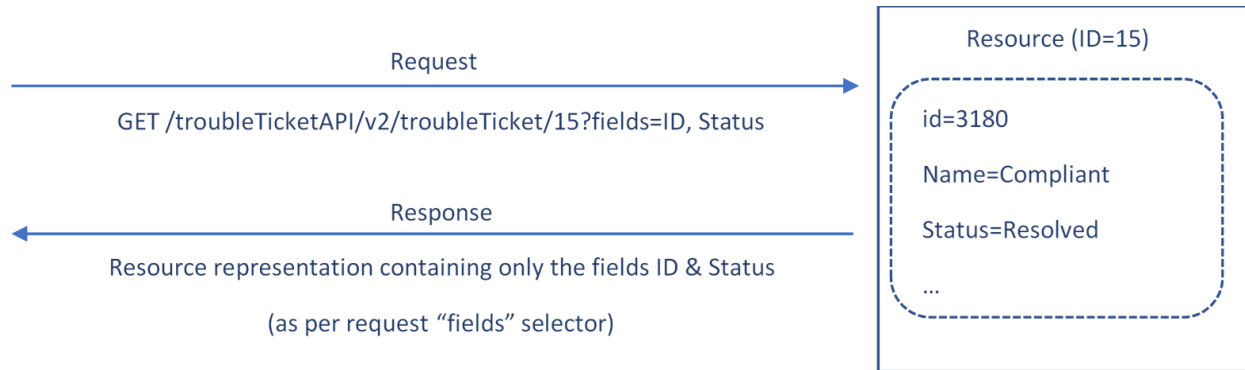


Figure 7: Example of a partial response mechanism

The existing **fields** selector is enhanced to support a JSONPath expression so that the API consumers have more flexibility in selecting parts of the resources that are in a multi-level array structure.

The `.(dot)` notation, specified in [part 1 of the Design Guideline](#), is the same notation used by JSONPath so the transition to the JSONPath expression will be transparent.

Chapter 2. JSONPath

2.1. What is it

In the XML world, the XPath (XML Path Language), standardized by the W3C provides the ability to select and extract data out of XML documents. The equivalent of XPath for JSON documents is called "JSONPath", defined by [\[RFC9535\]](#). It is a query-oriented language that allows querying and extraction of subsection(s) of a JSON document.

Every JSON document is based on a tree hierarchy of nodes (leaves), where every node is a JSON element which can be a simple leaf or complex one. The JSONPath language navigates this tree representation, selecting nodes through filtering criteria.

2.2. Example JSON Document

The following simplified example of a TMForum TroubleTicket JSON will be used as a reference Input Value throughout this document:

```
{
  "id": "3180",
  "href": "{apiRoot}/troubleTicket/v5/troubleTicket/3180",
  "name": "Complaint over last bill",
  "status": "Resolved",
  "relatedEntity": [
    {
      "id": "3472",
      "href": "{apiRoot}/customerBillManagement/v5/customerBill/3472",
      "name": "November Bill",
      "@referredType": "CustomerBill"
    },
    {
      "id": "3473",
      "href": "{apiRoot}/customerBillManagement/v5/customerBill/3473",
      "name": "December Bill",
      "@referredType": "CustomerBill"
    }
  ],
  "statusChange": [
    {
      "status": "Pending",
      "changeReason": "Need more information from the customer",
      "changeDate": "2018-05-01T00:00:00Z:00Z"
    }
  ],
}
```

```
{
  "status": "InProgress",
  "changeReason": "Working on the issue resolution",
  "changeDate": "2018-05-02T00:00:00Z"
},
{
  "status": "Resolved",
  "changeReason": "Issue has been resolved",
  "changeDate": "2018-05-02T00:00:00Z"
}
],
"note": [
  {
    "id": "1",
    "date": "2018-05-01T00:00:00Z",
    "author": "Mr John Wils",
    "text": "Missing necessary information from the customer"
  },
  {
    "id": "2",
    "date": "2018-05-01T00:00:00Z",
    "author": "Mr Erika Xavy",
    "text": "Information has been received, we are working on the resolution"
  },
  {
    "id": "3",
    "date": "2018-05-02T00:00:00Z",
    "author": "Mr Redfin Tekram",
    "text": "Issue has been resolved, the service has been restored"
  }
],
"attachment": [
  {
    "description": "Scanned disputed December bill",
    "href": "{apiRoot}/documentManagement/v5/attachment/44",
    "id": "44",
    "url": "https://xxxxx",
    "name": "December Bill",
    "size": 300,
    "sizeUnit": "KB",
    "@referredType": "Attachment"
  },
  {
    "description": "Scanned disputed November bill",
    "href": "{apiRoot}/documentManagement/v5/attachment/45",
```

```
    "id": "45",
    "url": "https://xxxxx",
    "name": "November Bill ",
    "size": 500,
    "sizeUnit": "KB",
    "@referredType": "Attachment"
  }
],
"channel": {
  "id": "8774",
  "name": "Self Service",
  "@type": "Channel"
}
}
```



All further examples are relative to the TroubleTicket entity presented in the JSON representation above.

A JSON document doesn't necessarily have a dedicated (named) element to represent the "root" of the structure but in the JSONPath case, a special notation has been introduced to represent the top-level element/root node.

The root node contains all the other JSON elements and based on our example some of them are simple nodes (**id**, **href**, **name**) and non-leaf ones – containing other nodes like (**relatedEntity**, **statusChange**, **note**).

JSONPath provides a uniform syntax (further details in the "syntax" chapter) to define expressions that can traverse a JSON document to extract the relevant subsections. For example, to retrieve the value for the "name" node of the root element, the following simple JSONPath query expression is used: **\$.name** (**\$** represents the root element). Much more complex paths and expressions can be used, that will allow traversal of the entire document looking for an indicated field, or to filter based on the field's values.

2.3. Syntax

JSONPath makes use of special notation and syntax to represent the nodes and links between them.



The JSONPath syntax and semantics is included here only for convenience. In case of contradictions or incompleteness, the specification in [\[RFC9535\]](#) takes precedence over the descriptions in this section, if not expressively stated otherwise.

A JSONPath expression starts with the top level identifier followed by a series of zero or more segments, each of which contains one or more selectors:

```
$[selector,selector,...][selector,selector,...]...
```



Overruling the respective requirement of [\[RFC9535\]](#), the leading **\$** can be omitted when used in the context of selectors for TMF API data structures.

The input value of a JSONPath expression is processed sequentially by each segment, where the output of one segment becomes the input to the next segment.

2.3.1. Identifiers

The *top-level element* of the JSON document represents the "root" element and is notated with the dollar sign **\$**.

The *current node* identifier **@** refers to the current node in the context of the evaluation of a JSONPath expression. It is mainly used in filter expressions to refer to the node being processed.

2.3.2. Segments

Segments address the direct children (**[]**) or any-level descendants (**[.. []**) of an input value.

2.3.3. Selectors

To select children or descendants addressed by a segment, one or more selectors are used between the brackets of the segment. Several selectors are separated by comma **,** and are combined with a logical OR, i.e. the result is the union of the results obtained by applying each of the selectors independantly from the others.

name selector

Selects the child resp. descendant element(s) of the input value with the specified name.

e.g. **['name']** selects the child resp. the descendants of the input value having the specified name; the name can be enclosed in single or double quotes.

If the name does not contain any special characters and starts with a character from A..Za..z_ the dot notation can be used as a shorthand:

e.g. `.name`

index selector

Selects a child of an array by its 0-based array index; does not apply to descendant segments.



Array indexes are 0-based, i.e. if the array has n elements, the first element has index `0` and the last element has index `$n-1$` .

A non-negative index value refers to the actual index (0-based, counting from the start of the array), whereas a negative value counts from the array end backwards, obtaining an equivalent non-negative index-selector by adding the length of the array to the negative index.

It is not an error when the index is out of range of the array, but simply nothing is selected.

e.g. `[0]` selects the first child of the input value if it is an array.

e.g. `[-1]` selects the last child (index $n-1$) of the input value if it is an array.

wildcard

Selects all children resp. descendants of the input value, regardless of their names or indexes.

e.g. `[*]`

Also, the shorthand notation can be applied here:

`.*`

array slice

`[start:end:step]` selects a range of children of an array, with possibly a step indicator; does not apply to descendant segments.



Array indexes are 0-based, i.e. if the array has n elements, the first element has index `0` and the last element has index `$n-1$` .

`step` including its leading colon may be omitted, indicating that all elements in the specified range are selected.

`start` may be omitted, indicating that the selection starts with the first element of the array.

end may be omitted, indicating that the selection ends with the last element of the array.

Positive values of **start** and **end** refer to the actual index (0-based, counting from the start of the array), whereas negative values count from the array end backwards, obtaining an equivalent non-negative index-selector by adding the length of the array to the negative index. For example, the selector -1 selects the last, and the selector -2 selects the penultimate element of an array with at least two elements.

A positive value of the **step** indicates increments, and a negative value denotes decrements, in both cases beginning at the position specified by the **start** value; a **step** or 0 does not select any element.

It is not an error when no element exists at a position indicated by these parameters, this simply means that this element is not contained in the selected set of elements.

e.g. **[0:7:3]** selects the first, fourth and seventh children of the input value if it is an array (starting with index 0, until index 7, in steps of 3).

e.g. **[::-1]** selects all elements in reverse order.

e.g. **[:10]** applied to an array having five elements selects all five elements.

filter expression

selects all children resp. descendants of the input value that match the specified filter expression; the filter expression starts with a question mark **?**, followed by a logical expression.

e.g. **[?@.name=='John']** selects all children resp. descendants of the input value that have a child element named **name** with the value **John**.

2.3.4. Logical Expressions

The logical expression used in a filter expression is evaluated for each child resp. descendant of the input value, and if the expression evaluates to true, the child resp. descendant is selected.

Basic logical expressions are comparisons between two operands and existence test.

Comparisons:

Comparisons Operators can be:

Operator	Description
<code>==</code>	Equality operator. The expression evaluates to true if the two operands are equal.
<code>!=</code>	Inequality operator. The expression evaluates to true if the two operands are not equal.
<code><</code>	Less-than operator. The expression evaluates to true if the left operand is less than the right operand.
<code>></code>	Greater-than operator. The expression evaluates to true if the left operand is greater than the right operand.
<code><=</code>	Less-than-or-equal operator. The expression evaluates to true if the left operand is less than or equal to the right operand.
<code>>=</code>	Greater-than-or-equal operator. The expression evaluates to true if the left operand is greater than or equal to the right operand.

Operands of a comparison can be:

literals

JSON string, JSON number, boolean (true/false), null

single-result queries

the root node `$` or the current node `@` followed by any number of segments with name- or index-selectors each selecting a child element of their input.

e.g. `'$.relatedEntity[0].name'` selects "November Bill", the value of the `name` attribute of the first element of the `relatedEntity` array.

e.g. `'@.size'` when the current element is the attachment with id 45, selects `500`, the value of the `size` attribute.

function-expression

function-expression that evaluates to a single JSON value or Nothing.

e.g. `length("some string")` evaluates to `11`.

Existence Tests:

An existence test is a possibly negated query or function expression by itself.

If the query resp. function selects at least one node, it yields `true`.

If the query resp. function does not select any node, it yields `false`.

When negated, the result is inverted.

Existence tests differ from comparisons in that:

- they work with arbitrary relative or absolute queries (not just singular queries),
- they work with queries that select structured values.

e.g. `!@.name` tests if the current element has no descendant element (on any level) named `name`.

e.g. `$.attachment` tests if the root element has a child element named `attachment` (it does not check if the attachment is an array and has any element).

e.g. `$.attachment[*]` tests if the root element has a child element named `attachment` which has any sub-element (which could be the case if attachment is an object having sub-elements, or if it is an array with at least one element).

e.g. `$.attachment[0]` tests if the root element has a child element named `attachment` which is an array with at least one element.

2.3.5. Logical Operators

Logical expressions can be combined to a larger logical expression using the logical operators AND `&&`, OR `||`, and NOT `!`, together with parenthesis `( )` to group sub-expressions:

Operators	Description
<code> </code>	Logical OR operator. The expression evaluates to true if at least one of the operands is true.
<code>&&</code>	Logical AND operator. The expression evaluates to true if both operands are true. The AND operator binds more tightly than the OR operator.
<code>!</code>	Logical NOT operator. The expression evaluates to true if the operand is false. The NOT operator binds more tightly than the AND, and OR operators.
<code>()</code>	Grouping operator. The expression evaluates to true if the operand is true. The grouping operator binds more tightly than the NOT, AND, and OR operators.

As usual, AND resp. OR can be chained to combine any number of sub-expressions without the need of parenthesis.

2.3.6. Function Extensions

JSONPath defines an extension point that can be used to implement "function extensions" as additional filter expression functionality.

A function extension is a registered unique name that can be applied to a sequence of zero or more arguments, producing a result. A function extension **MUST** be defined such that its evaluation is free of side effects, i.e., all possible orders of evaluation and choices of short-circuiting or full evaluation of an expression containing it **MUST** lead to the same result; memoization or logging are not side effects in this sense as they do not influence the result of the evaluation.

Each parameter and the result of a function extension must have a declared type. There exist the following types:

ValueType

JSON Values or *Nothing* (representing the absence of a JSON Value, distinct from any JSON Value including `null`)

LogicalType

LogicalTrue or *LogicalFalse* (unrelated to JSON Values `true` and `false`)

NodesType

Nodelists

2.3.7. Standard Function Extensions

An API implementation supporting JSONPath **MUST** support the following standard function extensions. It **MAY** also support further, non-standard function extensions.

length()

- one parameter of type *ValueType*
- result of type *ValueType* (unsigned integer or *Nothing*)
- if the parameter is a JSON string, the result is the number of Unicode characters in the string
- if the parameter is a JSON array, the result is the number of elements in the array
- if the parameter is a JSON object, the result is the number of members in the object
- for any other parameter value, the result is *Nothing*

count()

- one parameter of type *ValueType*

- result of type *ValueType* (unsigned integer or *Nothing*)
- results in the number of nodes in the parameter if it is a Nodelist, without removing of duplicates, and independent of the node's values.

match()

- first parameter of type *ValueType* (string)
- second parameter of type *ValueType* (string conforming to the regular expression format defined by [RFC9485](#))
- result of type *LogicalType*
- checks if the entire first parameter matches the regular expression given as second parameter

search()

- first parameter of type *ValueType* (string)
- second parameter of type *ValueType* (string conforming to the regular expression format defined by [RFC9485](#))
- result of type *LogicalType*
- checks if a substring of the first parameter matches the regular expression given as second parameter

value()

- one parameter of type *NodeType*
- result of type *ValueType*
- if the parameter is a single node, the result is the value of that node
- if the parameter is the empty Nodelist, the result is *Nothing*

2.4. Examples

The following examples illustrate the usage of JSONPath expressions on the TroubleTicket JSON structure provided in the [Example JSON Document](#) section.

JSONPath	Result
<code>\$.channel</code>	All the properties of the channel Output: <pre>{ "id": "8774", "name": "Self Service", "@type": "Channel" }</pre>
<code>\$('#id','name','href')</code>	Select multiple children from the root element. This operator is very useful in the partial response scenario. Output: <pre>{ "id": "3180", "name": "Complaint over last bill", "href": "{apiRoot}/troubleTicket/v5/troubleTicket/3180" }</pre>
<code>\$.name</code>	Select all the name attributes across all the resource structure. Output: <pre>["Complaint over last bill", "November Bill", "December Bill", "December Bill", "November Bill ", "Self Service"]</pre>

JSONPath	Result
<code>\$.note[*].author</code>	All the authors attribute values from all the elements of the 'note' array Output: <pre>["Mr John Wils", "Mr Erika Xavy", "Mr Redfin Tekram"]</pre>
<code>\$.note[*]</code>	Wildcard select for all the elements within the note array Output: <pre>[{ "id": "1", "date": "2018-05-01T00:00:00Z", "author": "Mr John Wils", "text": "Missing necessary information from the customer" }, { "id": "2", "date": "2018-05-01T00:00:00Z", "author": "Mr Erika Xavy", "text": "Information has been received, we are working on the resolution" }, { "id": "3", "date": "2018-05-02T00:00:00Z", "author": "Mr Redfin Tekram", "text": "Issue has been resolved, the service has been restored" }]</pre>

JSONPath	Result
<code>\$.note[1]</code>	Retrieve the second element of the note array (remember: indexes start from 0; does not consider the <code>id</code> attribute) Output: <pre>{ "id": "2", "date": "2018-05-01T00:00:00Z", "author": "Mr Erika Xavy", "text": "Information has been received, we are working on the resolution" }</pre>
<code>\$.note[0,2]</code>	Retrieve the first and third elements of the array (remember: indexes start from 0; does not consider the <code>id</code> attribute) Output: <pre>[{ "id": "1", "date": "2018-05-01T00:00:00Z", "author": "Mr John Wils", "text": "Missing necessary information from the customer" }, { "id": "3", "date": "2018-05-02T00:00:00Z", "author": "Mr Redfin Tekram", "text": "Issue has been resolved, the service has been restored" }]</pre>

JSONPath	Result
<code>\$.note[1:]</code>	Retrieve the elements from the array starting from position 1 till the end, i.e. all but the first element (remember: indexes start from 0) Output: <pre>[{ "id": "2", "date": "2018-05-01T00:00:00Z", "author": "Mr Erika Xavy", "text": "Information has been received, we are working on the resolution" }, { "id": "3", "date": "2018-05-02T00:00:00Z", "author": "Mr Redfin Tekram", "text": "Issue has been resolved, the service has been restored" }]</pre>

JSONPath	Result
<code>\$.note[:2]</code>	Selects the first 2 elements of the array (remember: indexes start from 0, and the element at the end index is not included). Output: <pre>[{ "id": "1", "date": "2018-05-01T00:00:00Z", "author": "Mr John Wils", "text": "Missing necessary information from the customer" }, { "id": "2", "date": "2018-05-01T00:00:00Z", "author": "Mr Erika Xavy", "text": "Information has been received, we are working on the resolution" }]</pre>

JSONPath	Result
<code>\$.note[-2:]</code>	Selects the last 2 elements of the array. Output: <pre>[{ "id": "2", "date": "2018-05-01T00:00:00Z", "author": "Mr Erika Xavy", "text": "Information has been received, we are working on the resolution" }, { "id": "3", "date": "2018-05-02T00:00:00Z", "author": "Mr Redfin Tekram", "text": "Issue has been resolved, the service has been restored" }]</pre>

JSONPath	Result
<code>\$.statusChange[?(@.status=='Pending')]</code>	Filter expression to select all the elements of the statusChange array where the status attribute is equal to 'Pending'. Output: <pre>[{ "status": "Pending", "changeReason": "Need more information from the customer", "changeDate": "2018-05-01T00:00:00Z" }]</pre> The <code>\$</code> represents the root element, while <code>statusChange</code> is the name of the root's child element, in the example an array. Each element of the array is processed by the filter selector. The <code>@</code> represents the current item being processed, and <code>status</code> is the name of the array items sub-element being compared to the literal. If the comparison does not validate, an empty response will be returned.
<code>statusChange[?(@.status==\$.status)]</code>	The comparison expression of the filter expression can also be based on the values outside the current object. Output: <pre>[{ "status": "Resolved", "changeReason": "Issue has been resolved", "changeDate": "2018-05-02T00:00:00Z" }]</pre>

JSONPath	Result
<code>\$.attachment[?(@.size==300)]</code>	Output: <pre>[{ "description": "Scanned disputed December bill", "href": "{apiRoot}/documentManagement/v5/attachment/44", "id": "44", "url": "https://xxxxx", "name": "December Bill", "size": 300, "sizeUnit": "KB", "@referredType": "Attachment" }]</pre>
<code>\$.statusChange[?(@.status!='Pending')]</code>	Output: <pre>[{ "status": "InProgress", "changeReason": "Working on the issue resolution", "changeDate": "2018-05-02T00:00:00Z" }, { "status": "Resolved", "changeReason": "Issue has been resolved", "changeDate": "2018-05-02T00:00:00Z" }]</pre>

JSONPath	Result
<code>\$.attachment[?(@.size>300)]</code>	Output: <pre>[{ "description": "Scanned disputed November bill", "href": "{apiRoot}/documentManagement/v5/attachment/45", "id": "45", "url": "https://xxxxx", "name": "November Bill ", "size": 500, "sizeUnit": "KB", "@referredType": "Attachment" }]</pre>

JSONPath	Result
<code>\$.attachment[?(@.size>=300)]</code>	Output: <pre>[{ "description": "Scanned disputed December bill", "href": "{apiRoot}/documentManagement/v5/attachment/44", "id": "44", "url": "https://xxxxx", "name": "December Bill", "size": 300, "sizeUnit": "KB", "@referredType": "Attachment" }, { "description": "Scanned disputed November bill", "href": "{apiRoot}/documentManagement/v5/attachment/45", "id": "45", "url": "https://xxxxx", "name": "November Bill ", "size": 500, "sizeUnit": "KB", "@referredType": "Attachment" }]</pre>

JSONPath	Result
<code>\$.attachment[?(@.size<301)]</code>	Output: <pre>[{ "description": "Scanned disputed December bill", "href": "{apiRoot}/documentManagement/v5/attachment/44", "id": "44", "url": "https://xxxxx", "name": "December Bill", "size": 300, "sizeUnit": "KB", "@referredType": "Attachment" }]</pre>
<code>\$.attachment[?(@.size<=300)]</code>	Output: <pre>[{ "description": "Scanned disputed December bill", "href": "{apiRoot}/documentManagement/v5/attachment/44", "id": "44", "url": "https://xxxxx", "name": "December Bill", "size": 300, "sizeUnit": "KB", "@referredType": "Attachment" }]</pre>

JSONPath	Result
<code>\$.attachment[?(!@.size)]</code>	Select all elements of the attachment array where the size attribute does not exist. Output: <pre>[]</pre> Nothing is retrieved because all the elements in the attachment array have an attribute called size.
<code>\$.attachment[?(@.size==300 && @.sizeUnit=='KB')]</code>	Select all elements of the attachment array where the size attribute is equal to 300 and the sizeUnit attribute is equal to 'KB'. Output: <pre>[{ "description": "Scanned disputed December bill", "href": "{apiRoot}/documentManagement/v5/attachment/44", "id": "44", "url": "https://xxxxx", "name": "December Bill", "size": 300, "sizeUnit": "KB", "@referredType": "Attachment" }]</pre>

JSONPath	Result
<code>\$.attachment[?(@.size==300 @.size==500)]</code>	Select all elements of the attachment array where the size attribute is equal to 300 or to 500. Output: <pre>[{ "description": "Scanned disputed December bill", "href": "{apiRoot}/documentManagement/v5/attachment/44", "id": "44", "url": "https://xxxxx", "name": "December Bill", "size": 300, "sizeUnit": "KB", "@referredType": "Attachment" }, { "description": "Scanned disputed November bill", "href": "{apiRoot}/documentManagement/v5/attachment/45", "id": "45", "url": "https://xxxxx", "name": "November Bill ", "size": 500, "sizeUnit": "KB", "@referredType": "Attachment" }]</pre>
<code>\$.note[?(length(@.text)>=100)]</code>	Select all elements of the note array where the length of the text attribute is greater than or equal to 100 characters. Output: <pre>[]</pre> Nothing is retrieved because none of the elements in the note array have a text attribute with a length greater than or equal to 100 characters.

JSONPath	Result
<code>\$.statusChange[?(match(@.status,/Resol.*?/i))]</code>	Select all elements of the statusChange array where the value of the status attribute matches the regular expression 'Resol.*' (case insensitive). Output: <pre>[{ "status": "Resolved", "changeReason": "Issue has been resolved", "changeDate": "2018-05-02T00:00:00Z" }]</pre>



- JSONPath expressions, including property names and values, are **case-sensitive**.
- Unlike XPath, JSONPath does not have operations for accessing parent or sibling nodes from the given node.

Chapter 3. URL / URI Encoding of the JSONPath expressions

The JSONPath syntax makes use of a set of special characters like `$`, `[`, `]`, `(`, `)`, `?`.

As the JSONPath will be passed as the value in one of the `filter` or `fields` selectors, special care must be taken in making sure the proper encoding is being used.

For this, the following arguments and standards have been taken into account:

- [\[RFC3986\]](#)

- **section 2.2: Reserved Characters**

- *URIs include components and subcomponents that are delimited by characters in the "reserved" set. These characters are called "reserved" because they may (or may not) be defined as delimiters by the generic syntax, by each scheme-specific syntax, or by the implementation-specific syntax of a URI's dereferencing algorithm. If data for a URI component would conflict with a reserved character's purpose as a delimiter [emphasis added], then the conflicting data must be percent-encoded before the URI is formed.*

- `reserved = gen-delims / sub-delims`

- `gen-delims = ":" / "/" / "?" / "#" / "[" / "]" / "@"`

- `sub-delims = "!" / "$" / "&" / "'" / "(" / ")"`
 `/ "*" / "+" / "," / ";" / "="`

- **section 3.3: Path Component**

- `pchar = unreserved / pct-encoded / sub-delims / ":" / "@"`

- **section 3.4: Query Component**

- `query = *( pchar / "/" / "?" )`

- **section 3.2.2: Host**

- *A host identified by an Internet Protocol literal address, version 6 [\[RFC3513\]](#) or later, is distinguished by enclosing the IP literal within square brackets ("`[`" and "`]`"). This is the only place where square bracket characters are allowed in the URI syntax.*

- **appendix A: Collected ABNF for URI**

- `pchar = unreserved / pct-encoded / sub-delims / ":" / "@"`

- `pct-encoded = "%" HEXDIG HEXDIG`

- `unreserved = ALPHA / DIGIT / "-" / "." / "_" / "~"`

□ sub-delims = "!" / "\$" / "&" / "'" / "(" / ")" / "*" / "+" / "," / ";" / "="

- [\[TMF630-1\]](#):

The TM Forum Design Guideline already allows for the ; and for the , in the query segment of the URI to be used without being escaped.

As such only the [and] characters will be encoded.

Decoded:

```
GET {apiRoot}/troubleTicket/?filter=attachment[?(@.size==300)]
```

Encoded:

```
GET {apiRoot}/troubleTicket/?filter=attachment%5B?(@.size==300)%5D
```



For readability purpose the examples of JSONPath expression in the rest of the document don't have the characters [and] in their encoded form.

Chapter 4. Collection filtering using JSONPath

This pattern describes how to retrieve a subset of resources from a collection through an extended query filtering mechanism. In Design Guidelines Part 1 ([[TMF630-1](#)]) the filtering is done using the name/value query parameters on the entity attributes.

This part of the design guide enhances the query design pattern by defining a JSONPath attribute(s) selector that will enhance the existing capabilities for the entity attributes that are arrays. The JSONPath predicate expression will allow the selection of all the resources that will match the predicate condition.



All examples are relative to the management of the TroubleTicket entity having the same JSON representation as in the above chapter.



For better readability, JSONPath expressions in the query part of URLs are shown in their non-encoded form. In real applications, encoding is necessary.



Figure 8: Example of the filtering mechanism

The enhancement of the filtering mechanism is done by defining a **filter** query parameter that will act as a selector:

```
?filter={JSONPath Expression}
```

Unlike the basic filtering, the *filter* selector takes as a parameter a JSONPath predicate expression. As long as the predicate condition validates for a particular resource it will be returned in the response to the client.

Rule: An attribute selector directive called **filter** MUST be used to specify the JSONPath expression.

Rule: Entity attribute selection via JSONPath expression MAY optionally be enabled.

Rule: In case the JSONPath **filter** selector is not supported a **501 Not Implemented** (see [\[RFC9110\]](#) section 15.6.2) MUST be returned,

The filtering expression is a sequence of JSONPath predicates that will perform assertions at the resource's attribute level.

```
GET {apiRoot}/{resourceName}?filter={JSONPath Expression}
```

For example:

```
GET {apiRoot}/troubleTicket?filter=attachment[?(@.size==300)]
```



In the above example the leading **\$.** was omitted.

Rule: In the JSONPath expression from the **filter** selector the leading **\$.** MAY be omitted.

The **filter** will select and return all the TroubleTicket resources that have in the **attachment** array elements where the attribute **size** is equal to 300. The **filter** selector can also be combined with the standard query parameters forming a more complex query expression.

For example, the following query will return all the "TroubleTicket" resources that have **status=resolved** and that have in the **attachment** array, elements where the attribute **size** is equal to 300:

```
GET {apiRoot}/troubleTicket?status=resolved&filter=attachment[?(@.size==300)]
```

Rule: The complete resource representations (with all the attributes) of all the matching entities must be returned.

Rule: The returned representation of each entity must contain a field called **id** and that field must be populated with the resourceID.

Rule: If the request is successful then the returned code MUST be **200 OK**. Exceptions must use the exception codes from [\[IANA-STATUS\]](#).

Rule: If the JSONPath expression that is being passed in the **filter** selector is invalid a **400 Bad Request** (see [\[RFC9110\]](#) section 15.5.1) MUST be returned.

Rule: The subset obtained through the use of the **filter** selector is under the same

pagination rules as described in the [Design Guideline Part 1](#).

For example:

```
GET {apiRoot}/troubleTicket?offset=10&limit=20&filter=attachment[?(@.size==300)]
```

will retrieve the twenty resources starting at the tenth where the **attachment** array has the element **size** equal with 300.

JSONPath filter ORING is also supported and is achieved following the equivalent pattern

```
[filter={JSONPath Expression},{JSONPath Expression}*]
```

of the type described in [\[TMF630-1\]](#). ORING can be used many times as required.

For example:

```
GET {apiRoot}/troubleTicket?status=resolved&filter=attachment[?(@.sizeUnit=='KB' && @.size==500)],attachment[?(@.sizeUnit=='MB' && @.size==0.5)]
```

ORing can also be achieved by using **;** again as an equivalent pattern to the one described in [\[TMF630-1\]](#):

```
[filter={JSONPath Expression};filter={JSONPath Expression}*]
```

The default behavior is to return all resources where the JSONPath predicate is a match. As such simple filters like: `[?(@)]`, which is return true for all the resource members will result in selecting all the resources in the collection.

The basic filtering can be combined with the **filter** selector. When both mechanisms are used, the result set must be a match for both conditions. For example:

Request:

```
GET {apiRoot}/troubleTicket?status=resolved&filter=attachment[?(@.sizeUnit=='KB' && @.size==500)]
```

Response:

200 OK

Content-Type: application/json

```
[
  {
    "id": "3180",
    "href": "{apiRoot}/troubleTicket/v5/troubleTicket/3180",
    "name": "Complaint over last bill",
    "status": "resolved",
    "relatedEntity": [
      {
        "id": "3472",
        "href": "{apiRoot}/customerBillManagement/v5/customerBill/3472",
        "name": "November Bill",
        "@referredType": "CustomerBill"
      },
      {
        "id": "3473",
        "href": "{apiRoot}/customerBillManagement/v5/customerBill/3473",
        "name": "December Bill",
        "@referredType": "CustomerBill"
      }
    ],
    "statusChange": [
      {
        "status": "Pending",
        "changeReason": "Need more information from the customer",
        "changeDate": "2018-05-01T00:00:00Z"
      },
      {
        "status": "InProgress",
        "changeReason": "Working on the issue resolution",
        "changeDate": "2018-05-02T00:00:00Z"
      },
      {
        "status": "Resolved",
        "changeReason": "Issue has been resolved",
        "changeDate": "2018-05-02T00:00:00Z"
      }
    ],
    "note": [
      {
        "id": "1",
```

```
    "date": "2018-05-01T00:00:00Z",
    "author": "Mr John Wils",
    "text": "Missing necessary information from the customer"
  },
  {
    "id": "2",
    "date": "2018-05-01T00:00:00Z",
    "author": "Mr Erika Xavy",
    "text": "Information has been received, we are working on the resolution"
  },
  {
    "id": "3",
    "date": "2018-05-02T00:00:00Z",
    "author": "Mr Redfin Tekram",
    "text": "Issue has been resolved, the service has been restored"
  }
],
"attachment": [
  {
    "description": "Scanned disputed December bill",
    "href": "{apiRoot}/documentManagement/v5/attachment/44",
    "id": "44",
    "url": "https://xxxxx",
    "name": "December Bill",
    "size": 300,
    "sizeUnit": "KB",
    "@referredType": "Attachment"
  },
  {
    "description": "Scanned disputed November bill",
    "href": "{apiRoot}/documentManagement/v5/attachment/45",
    "id": "45",
    "url": "https://xxxxx",
    "name": "November Bill ",
    "size": 500,
    "sizeUnit": "KB",
    "@referredType": "Attachment"
  }
],
"channel": {
  "id": "8774",
  "name": "Self Service",
  "@type": "Channel"
}
},
```

```
{
  "id": "3181",
  "href": "{apiRoot}/troubleTicket/v5/troubleTicket/3181",
  "name": "Complaint over last bill",
  "status": "Resolved",
  "relatedEntity": [
    {
      "id": "3473",
      "href": "{apiRoot}/customerBillManagement/v5/customerBill/3472",
      "name": "November Bill",
      "@referredType": "CustomerBill"
    }
  ],
  "statusChange": [
    {
      "status": "Pending",
      "changeReason": "Need more information from the customer",
      "changeDate": "2018-05-01T00:00:00Z"
    },
    {
      "status": "InProgress",
      "changeReason": "Working on the issue resolution",
      "changeDate": "2018-05-02T00:00:00Z"
    },
    {
      "status": "Resolved",
      "changeReason": "Issue has been resolved",
      "changeDate": "2018-05-02T00:00:00Z"
    }
  ],
  "note": [
    {
      "id": "3",
      "date": "2018-05-02T00:00:00Z",
      "author": "Mr Redfin Tekram",
      "text": "Issue has been resolved, the service has been restored"
    }
  ],
  "attachment": [
    {
      "description": "Scanned disputed December bill",
      "href": "{apiRoot}/documentManagement/v5/attachment/44",
      "id": "44",
      "url": "https://xxxxx",
      "name": "December Bill",
    }
  ]
}
```

```
        "size": 300,  
        "sizeUnit": "KB",  
        "@referredType": "Attachment"  
    },  
    ],  
    "channel": {  
        "id": "8774",  
        "name": "Self Service",  
        "@type": "Channel"  
    }  
}  
]
```

Chapter 5. Partial resource representation using JSONPath

This pattern describes how to retrieve a subset of the attributes from an entity in the response using an attribute filtering mechanism. In the current Design Guideline Part 1 the filtering is done using the selector directive called **fields** - [TMF630-1].

This part of the design guide enhances the **fields** selector to support a JSONPath (path or predicate).



All the rules described in [TMF630-1] will continue to apply.

Rule: In order to retrieve a partial representation using a JSONPath, the **fields** selector MUST be used:

```
GET {apiRoot}/{resourceName}/{resourceID}?fields={JSONPath*}
```

For example:

The request is to return only the **name** attribute from the troubleTicket entity.



as per [TMF630-1] the **id** and **href** of the resource will always be returned (where available).

Request:

```
GET {apiRoot}/troubleTicket/42?fields=channel.name
```

Response:

```
200 OK
Content-Type: application/json
```

```
{
  "id": "42",
  "channel": {
    "name": "Self Service"
  }
}
```

```
}
```

Another example, to retrieve only the attributes from the "note" array that match a particular predicate condition (the author is Mr. John Wils)

Request:

```
GET {apiRoot}/troubleTicket/42?fields=note[?(@.author=='Mr John Wils')]
```

Response:

```
200 OK
Content-Type: application/json
```

```
{
  "id": "1",
  "date": "2018-05-01T00:00:00Z",
  "author": "Mr John Wils",
  "text": "Missing necessary information from the customer"
}
```

Example to retrieve only specific JSON elements (simple or arrays):

Request:

```
GET {apiRoot}/troubleTicket/42?fields=['id','href','name','note']
```

Response:

```
200 OK
Content-Type: application/json
```

```
{
  "id": "3180",
  "href": "{apiRoot}/troubleTicket/v5/troubleTicket/3180",
  "name": "Complaint over last bill",
  "note": [
    {
```

```

    "id": "1",
    "date": "2018-05-01T00:00:00Z",
    "author": "Mr John Wils",
    "text": "Missing necessary information from the customer"
  },
  {
    "id": "2",
    "date": "2018-05-01T00:00:00Z",
    "author": "Mr Erika Xavy",
    "text": "Information has been received, we are working on the resolution"
  },
  {
    "id": "3",
    "date": "2018-05-02T00:00:00Z",
    "author": "Mr Redfin Tekram",
    "text": "Issue has been resolved, the service has been restored"
  }
]
}

```

JSONPath **fields** ORing is also supported and is achieved following the equivalent pattern:

```
?fields={JSONPath Expression},{JSONPath Expression}*
```

ORing can be used many times as required.

Example:

Request:

```

GET
{apiRoot}/troubleTicket?status=Resolved&fields=['id','href','name','note'],channel,note[?(@.author=='Mr John Wils')]

```

Response:

```

200 OK
Content-Type: application/json

```

```
{
```

```
"id": "3180",
"href": "{apiRoot}/troubleTicket/v5/troubleTicket/3180",
"name": "Compliant over last bill",
"channel": {
  "id": "8774",
  "name": "Self Service",
  "@type": "Channel"
},
"note": {
  "id": "1",
  "date": "2018-05-01T00:00:00Z",
  "author": "Mr John Wils",
  "text": "Missing necessary information from the customer"
}
}
```

Chapter 6. Sorting selector

This pattern describes how to request a particular sorting criterion. The sort directive is described in [\[TMF630-1\]](#). This part of the design guide enhances the "Sort-Field" selector to support a JSONPath.

The following specification is described in [\[TMF630-1\]](#) – *Sorting directive*:

Sort-Query-Parameters:

`sort=#( [ {Sort-Direction} ] {Sort-Field} )`

Sort-Direction:

`- | +`

Sort-Field:

The field to sort on; can be identified by a JSONPath expression.

(see the [Pattern Notation](#) described above)

Example:

```
GET {apiRoot}/troubleTicket?sort=channel.name
```

or

```
GET {apiRoot}/troubleTicket?sort=attachment[*].name
```

The sorting selector can always be combined with **filter** or **fields** selectors. For example:

```
GET  
{apiRoot}/troubleTicket?filter=attachment[?(@.size==300)]&fields==['id','href','name']&sort=attachment[*].name
```

Chapter 7. Notification Pattern – Registering Listener

The Notification Pattern and the mechanism of registering a listener are described in [TMF630-1]. This part of the design guide enhances the "query" expression that can be passed when registering a listener to support a JSONPath expression.

The query expression may be used to filter specific event types and/or any content of the event.

Considering the following example of resource:

```
{
  "id": "3180",
  "href": "{apiRoot}/troubleTicket/v5/troubleTicket/3180",
  "name": "Complaint over last bill",
  "status": "Resolved"
}
```

The structure of the event is:

```
{
  "eventId": "eventId",
  "eventTime": "eventTime",
  "eventType": "event Type",
  "event": {
    "resource": {
      "id": "3180",
      "href": "{apiRoot}/troubleTicket/v5/troubleTicket/3180",
      "name": "Complaint over last bill",
      "status": "Resolved"
    }
  }
}
```

Registering a notification with a **filter** selector is:

```
POST {apiRoot}/hub
Accept: application/json
```

```
{  
  "callback": "http://in.listener.com",  
  "query": "filter=[?(@.event.resource.status=='Resolved')]"  
}
```

Chapter 8. Error handling

The REST APIs MUST use the exception and response codes documented at [\[IANA-STATUS\]](#).

The same rules as described in [\[TMF630-1\]](#) will also apply for the cases where JSONPath is being used as a filtering mechanism.

Rule: If the JSONPath expression used in the **filter** or **fields** selectors fails at the syntax level, a **400 Bad Request** (see [\[RFC9110\]](#) section 15.5.1) MUST be returned.

Rule: If the JSONPath is not supported by the server implementation, but the client is making use of it in either **filter** or **fields** selectors, a **501 Not Implemented** (see [\[RFC9110\]](#) section 15.6.2) MUST be returned.

The syntax errors can be enhanced further through user and application-specific error codes via the mechanism described in the [\[TMF630-1\]](#).

In the example below, the JSONPath expression from the request is missing a **)** in the predicate:

Request:

```
GET {apiRoot}/troubleTicket?filter=[?(@.status=='Resoslved')]&fields=name
```

Response:

```
400 Bad Request
Content-type: application/json
```

```
{
  "code": "ERR001",
  "reason": "Invalid JSONPath expression present in the filter selector",
  "message": " Could not parse token starting at position 2. Expected ?, ', 0-9,
  *"
}
```

Chapter 9. JSON Patch

This part of the design guide enhances the existing JSON Patch extension to manage arrays, described in [\[TMF630-5\]](#) with the JSONPath capabilities.

The following solution is a deviation from the JSON Patch standard where the **path** element is a JSON Pointer, however, the deviation exists as described in [\[TMF630-5\]](#).

The existing proposal has the scope to extend the "selector" conditions.

It is always recommended to use the **test** operation before applying any patch.

Considering a troubleTicket resource such as {apiBase}/troubleTicket/v5/troubleTicket/1:

```
{
  "id": "1",
  "note": [
    {
      "date": "2013-07-25T06:55:12.0Z",
      "author": "John Doe",
      "status": "Edited"
    },
    {
      "date": "2013-07-24T09:55:30.0Z",
      "author": "Arthur Evans",
      "status": "Edited"
    },
    {
      "date": "2013-07-25T08:55:12.0Z",
      "author": "Mark Evens",
      "status": "Archived"
    }
  ]
}
```

The existing specification enables a JSON Patch document such as following where the condition is based on a simple path:

```
[
  {
    "op": "add",
    "path": "/note?note.author='John Doe'",
    "value": "Informed"
  }
]
```

```
}  
]
```

The JSONPath extension enhances the query and filtering capabilities.

Example – add where the "note" array has an "author" called John Doe

```
[  
  {  
    "op": "add",  
    "path": "note[?(@.author=='John Doe')]",  
    "value": {"text":"Informed"}  
  }  
]
```

Applying the above path to the sample JSON will result in the following path to be automatically computed: `$['note'][0]`

Example: add where the **author** is "John Doe" and the **status** is "Edited"

```
[  
  {  
    "op": "add",  
    "path": "note[?(@.author=='John Doe' && @.status=='Edited')]",  
    "value": {"text":"Informed"}  
  }  
]
```

All the other capabilities are also supported through the JSONPath mechanisms either by a simple path, by a predicate expression or by a combination of them:

Removing or replacing one of the components of an array

Example:

```
[  
  {  
    "op": "remove",  
    "path": "note[?(@.author=='John Doe')]"  
  }  
]
```

Removing or replacing an attribute from one of the components of an array

Example:

```
[  
  {  
    "op": "remove",  
    "path": "note[?(@.author=='John Doe')].date"  
  }  
]
```

References

- [E4X]
ECMAScript for XML
ECMA (European Computer Manufacturers Association)
<http://en.wikipedia.org/wiki/E4X>
- [IANA-STATUS]
Hypertext Transfer Protocol (HTTP) Status Code Registry
iana (Internet Assigned Numbers Authority)
<http://www.iana.org/assignments/http-status-codes/http-status-codes.xml>
- [JAYWAY]
Introduction to JsonPath
Baeldung – Tarnum Java SRL
<https://www.baeldung.com/guide-to-jayway-jsonpath>
- [MDN-REGEX]
Regular Expressions
Mozilla Developer Network, Mozilla Foundation
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions
- [RFC3513]
Internet Protocol Version 6 (IPv6) Addressing Architecture
IETF (Internet Engineering Task Force)
<https://www.ietf.org/rfc/rfc3513.html>
- [RFC3986]
Uniform Resource Identifier (URI): Generic Syntax
IETF (Internet Engineering Task Force)
<https://www.ietf.org/rfc/rfc3986.html>
- [RFC5234]
Augmented BNF for Syntax Specifications: ABNF
IETF (Internet Engineering Task Force)
<https://www.ietf.org/rfc/rfc5234.html>
- [RFC8174]
Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words
IETF (Internet Engineering Task Force)
<https://www.ietf.org/rfc/rfc8174.html>
- [RFC8259]
The JavaScript Object Notation (JSON) Data Interchange Format
IETF (Internet Engineering Task Force)
<https://www.ietf.org/rfc/rfc8259.html>

- [\[RFC9110\]](#)
HTTP Semantics
IETF (Internet Engineering Task Force)
<https://www.ietf.org/rfc/rfc9110.html>
- [\[RFC9535\]](#)
JSONPath – Query Expressions for JSON
IETF (Internet Engineering Task Force)
<https://www.ietf.org/rfc/rfc9535.html>
- [\[TMF630-1\]](#)
TMForum
REST API Design Guidelines
Part 1: Practical guidelines for RESTful APIs naming, CRUD, filtering, tasks, history, notifications
- [\[TMF630-2\]](#)
TMForum
REST API Design Guidelines
Part 2: Advanced guidelines for RESTful APIs polymorphism, extension patterns, depth and expand directive, entity RefOrValue
- [\[TMF630-3\]](#)
TMForum
REST API Design Guidelines
Part 3: Guidelines for extending TMF Open API's with hypermedia support
- [\[TMF630-4\]](#)
TMForum
REST API Design Guidelines
Part 4: Advanced guidelines for import and export tasks and for entity versioning
- [\[TMF630-5\]](#)
TMForum
REST API Design Guidelines
Part 5: JSON patch extensions to manage arrays
- [\[TMF630-6\]](#)
TMForum
REST API Design Guidelines
Part 6: JSONPath extensions
- [\[TMF630-7\]](#)
TMForum
REST API Design Guidelines
Part 7: JSON Schemas guideline

Administrative Appendix

This Appendix provides additional background material about the TM Forum and this document. In general, sections may be included or omitted as desired; however, a Document History must always be included.

Document History

Version History

This section records the changes between this and the previous document version as it is edited by the team concerned.



this is an incremental number which does not have to match the release number and used for change control purposes only.

Version Number	Date Modified	Modified by	Description of changes
1.0		Florin Tene	Initial version
1.1		Florin Tene	V5 update

Release History

This section records the changes between this and the previous Official document release. The release number is the 'Marketing' number which this version of the document is first being assigned to.

Release Number	Date Modified	Modified by	Description of changes

Acknowledgments

This document was prepared by the members of the TM Forum team:

Contributors to Prior Versions

Previous versions of this document were prepared by the following members of the TM Forum API team (in alphabetical order):

Florin Tene	Cityfibre	Contributor
-------------	-----------	-------------

Pierre Gauthier	TM Forum	Chief Architect, Team Leader
-----------------	----------	------------------------------

Contributors to Current Version

The following members of the TM Forum API team contributed to this version of the document (in alphabetical order):

Florin Tene	Cityfibre	Contributor
Lutz Bettge	Deutsche Telekom	Contributor, Editor